Langages et algorithmique	Chap 05	Fiche exercices

5.1 Préambule

Pour ces exercices, les fonctions seront à écrire dans un fichier Python sur VS Code.
On considère que l'alphabet des machines utilisées est :

$$\{0, 1, \texttt{None}\}$$

La valeur `None` représente une case vide du ruban.
Dans les exercices suivants :

- un entier binaire est représenté par une liste Python,
- la tête de lecture commence sur la première case du nombre.
- on utilise une variable `etat` pour représenter les états de la machine en Python.

5.2 Machine qui met les bits à zéro

On souhaite représenter une machine de Turing qui remplace tous les `1` par des `0` et ne modifie pas les `0`.
La machine s'arrête lorsqu'elle rencontre une case vide après avoir parcouru le nombre.
Exemple :

$$01010001 \rightarrow 00000000$$

Exercice 1 — Machine mettre à zéro

1. Lire le préambule (5.1)
2. Dessiner le diagramme des états de la machine.
3. Donner la table de transitions.
4. Écrire une fonction Python `mettre_a_zero(ruban)` qui simule cette machine de Turing.

5.3 Tester la parité d'un entier

On souhaite construire une machine de Turing qui teste si un entier écrit en binaire est **pair**.
La machine devra effacer tout le nombre puis écrire sur le ruban :

— `1` si l'entier est pair

— `0` sinon.

Exercice 2 — Tester si un entier est pair

1. Dessiner le diagramme des états.
2. Donner la table de transitions.
3. Écrire une fonction Python `est_pair(ruban)` qui simule cette machine.

5.4 Ajouter 1 à un nombre binaire

On souhaite construire une machine de Turing qui ajoute 1 à un entier écrit en binaire.
Exemples :

$$101 + 1 = 110$$

$$111 + 1 = 1000$$

Exercice 3 — Ajouter 1

1. Dessiner le diagramme des états.
2. Donner la table de transitions.
3. Écrire la fonction Python :`ajouter_un(ruban)`

5.4.1 Multiplier par 2

On souhaite construire une machine de Turing qui multiplie par 2 un entier écrit en binaire.

Exercice 4 — Multiplier par 2

1. Dessiner le diagramme des états.
2. Donner la table de transitions.
3. Écrire la fonction Python : `multiplier_par_deux(ruban)`

5.4.2 Soustraire 1

On souhaite construire une machine de Turing qui soustrait 1 à un entier binaire.
Exemples :

$$1101 - 1 = 1100$$

$$1000 - 1 = 0111$$

Exercice 5 — Soustraire 1

1. Dessiner le diagramme des états.
2. Donner la table de transitions.
3. Écrire la fonction Python : `soustrait_un(ruban)`

5.4.3 BONUS : Palindrome

On souhaite construire une machine de Turing qui détecte si un mot binaire est un palindrome.

Exercice 6 — Palindrome

1. Dessiner le diagramme des états.
2. Donner la table de transitions.
3. Écrire la fonction Python : `est_palindrome(ruban)`

Exercice 7 — Bac Métropole 2024 — Exercice 1 (6 points)

Cet exercice porte sur l'exécution d'un programme Python et sur la décidabilité.

Dans cet exercice, on dira qu'un appel `f(x)`, où `f` est une fonction Python prenant un argument et `x` est une valeur, **termine** lorsque l'évaluation de `f(x)` renvoie toujours une valeur au bout d'un nombre fini d'étapes. À l'opposé, un tel appel ne termine pas s'il est possible qu'il effectue des instructions à l'infini.

Partie A : boucle `while`

Commençons par un premier exemple, avec une fonction prenant un entier en argument et utilisant une boucle `while`.

```
def f1(n):
    i = n
    while i != 10:
        i = i + 1
    return i
```

1. Sur votre copie, donner les valeurs successives de la variable `i` lors de l'exécution de `f1(7)`, et indiquer si `f1(7)` termine.
2. Indiquer si l'appel `f1(-2)` se termine. Si oui, indiquer la valeur renvoyée.
3. Sur votre copie, donner les 5 premières valeurs prises par la variable `i` lors de l'exécution de `f1(12)`, et indiquer si l'appel `f1(12)` va terminer ou non.
4. Préciser pour quels entiers `n` l'appel `f1(n)` se termine.

Partie B : fonction récursive

Prenons maintenant un deuxième exemple, avec une fonction récursive (prenant elle aussi un entier en argument).

```
def f2(n):
    if n == 0:
        return 0
    else:
        return n + f2(n-2)
```

5. L'appel `f2(4)` termine-t-il ? Si oui, indiquer la valeur renvoyée par `f2(4)` ; sinon, justifier brièvement.
6. L'appel `f2(5)` termine-t-il ? Si oui, indiquer la valeur renvoyée par `f2(5)` ; sinon, justifier brièvement.
7. Déterminer l'ensemble des entiers naturels `n` pour lesquels l'appel `f2(n)` termine.
8. Écrire une fonction Python `infini`, récursive, telle que l'appel `infini(n)` ne termine pour aucun entier `n`.

Partie C : le problème de l'arrêt

On se demande maintenant s'il est possible d'écrire une fonction `arret` qui prend en arguments une chaîne de caractères `code_f` contenant le code d'une fonction `f` et un argument `x` de `f`, et tel que `arret(code_f, x)` renvoie `True` si l'appel `f(x)` va terminer et `False` sinon.

Dans la suite de cet exercice, on suppose disposer d'une telle fonction `arret` et on implémente la fonction suivante, utilisant cette fonction `arret`, ainsi que la fonction `infini` de la question précédente dont l'appel `infini(n)` ne termine jamais quelle que soit la valeur de `n`.

```
def paradoxe(x):
    if arret(x, x):
        infini(42)
    else:
        return 0
```

De même, on suppose disposer d'une variable `code_paradoxe` contenant le code de la fonction `paradoxe` sous la forme d'une chaîne de caractères, et on s'intéresse à l'appel `paradoxe(code_paradoxe)`.

Cet appel de fonction commence par effectuer le test `arret(code_paradoxe, code_paradoxe)` dans le `if` de la ligne 2.

9. Dans le cas où `arret(code_paradoxe, code_paradoxe)` renvoie `True`, préciser la prochaine instruction à être exécutée. Dans ce cas, expliquer si l'appel `paradoxe(code_paradoxe)` termine.
10. Dans le cas où `arret(code_paradoxe, code_paradoxe)` renvoie `False`, préciser la prochaine instruction à être exécutée. Dans ce cas, expliquer si l'appel `paradoxe(code_paradoxe)` termine.
11. En déduire qu'une telle fonction `arret` ne peut exister.